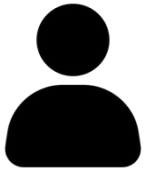
AWS Identity and Access Management

Cloud Security

CSE4059

Recap



Client
Requests a report

Web Application

EC2



S3
Stores reports

- Network filtering: Controls which connections are allowed
- TLS: Protects communication
- IAM: Controls access to AWS resources

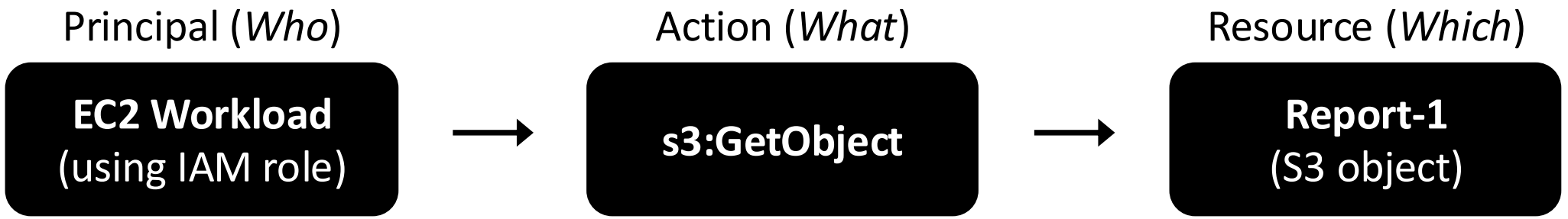
- How does AWS decide whether a request is allowed or denied?

AWS Identity and Access Management



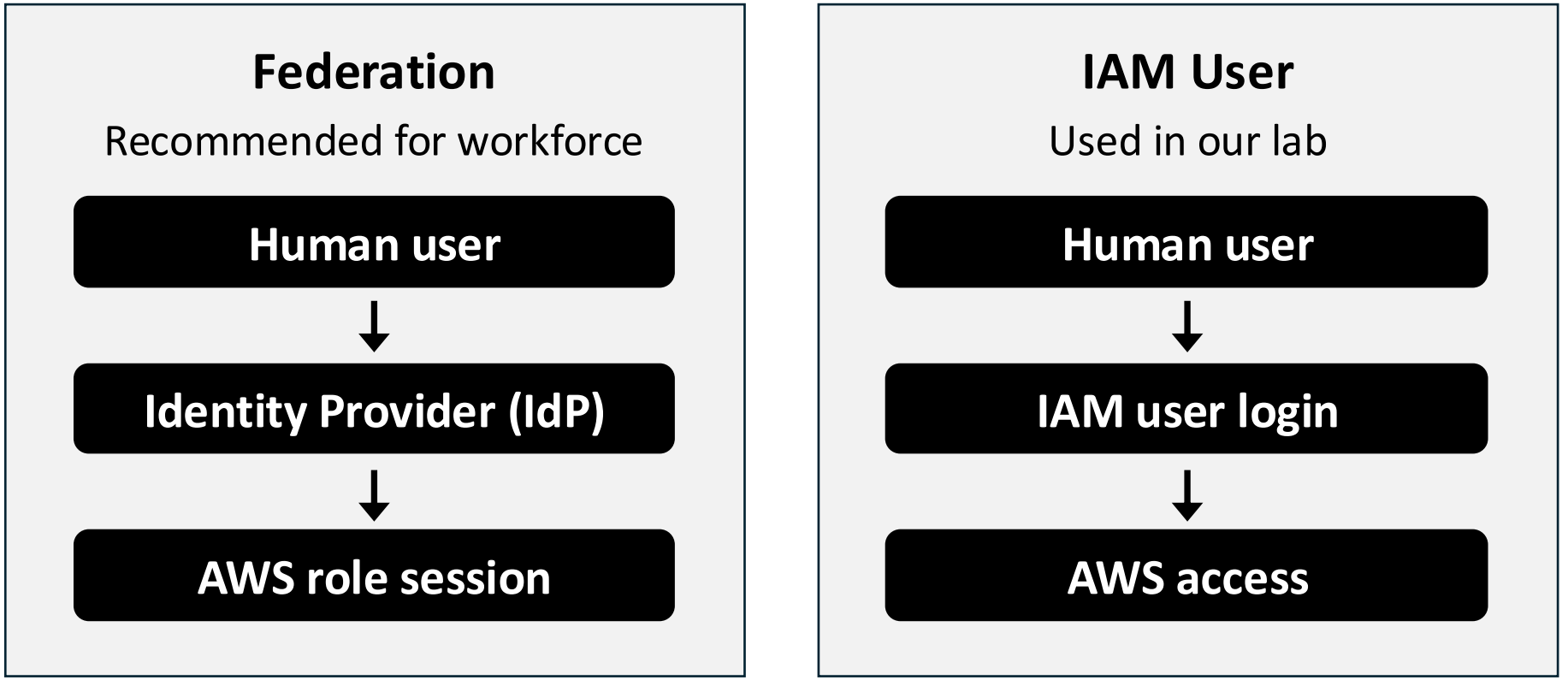
- AWS Identity and Access Management (IAM) manages access to AWS resources
  - Identities represent callers; policies define permissions
- An AWS request concerns an action on a resource
  - AWS evaluates the applicable policies before performing the operation
- IAM does not replace authorization inside the application
  - The application still decides which reports its users may receive

Access Requests



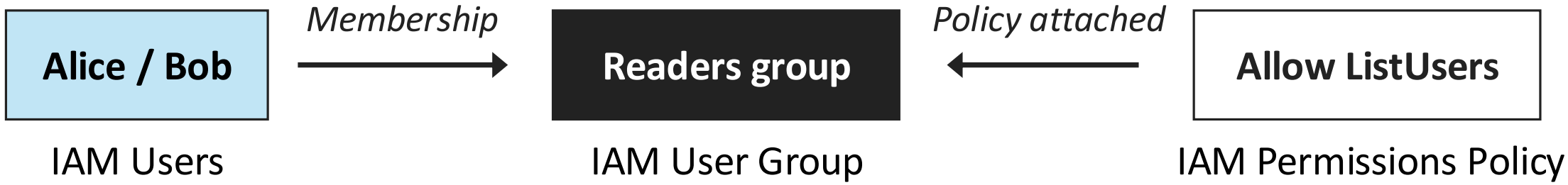
- A **principal** is the identity represented by a request
  - Here, the caller is a workload using an EC2 role
- The **action** identifies the operation / the **resource** identifies its target
  - **s3:GetObject** requests the contents of the **report** object
- Changing any of these elements can change the permission decision
  - Reading report-1 and deleting report-1 are different requests

Human Access to AWS



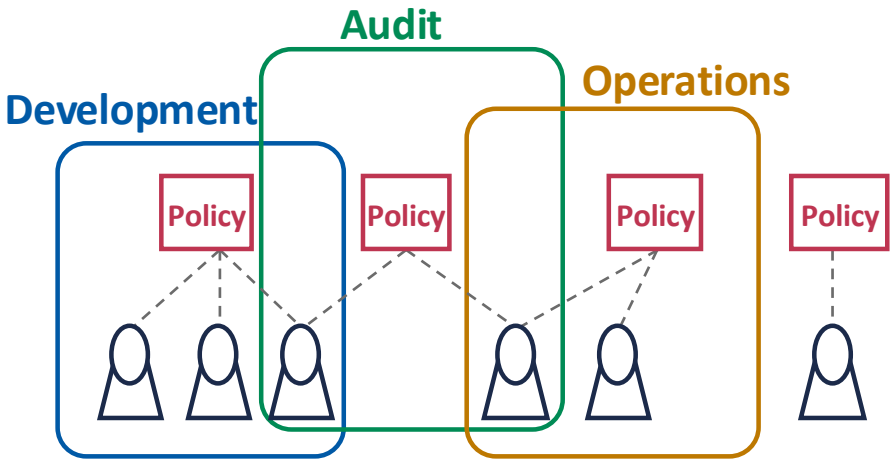
- AWS recommends federation and temporary credentials for workforce access

Users, Groups, and Policies



- An IAM user represents an identity within the AWS account
- A user group collects IAM users that need common permissions
  - The group does not sign in or send requests as a separate identity
- A policy is a document containing permission rules
  - Attaching a policy to a group makes its permissions available to the group's users
- Users remain distinct identities even when they receive the same permissions

User Groups



Overlaps: shared membership

Dashed lines: policy permissions available to users

- Groups organize users by duties, such as development, audit, and operations
  - The overlapping regions represent users with more than one membership
- A user can receive permissions from multiple groups or a direct policy
- Changing a group policy affects the permissions supplied through that group
- Removing one membership removes that permission source
  - Other group or direct grants must still be considered

Changing Group Membership



- Assume Readers is Alice's only source of *iam:ListUsers* permission
  - No explicit deny or additional restriction applies in this example
- With membership, the group policy supplies an Allow for the user-list request
- After removal, that Allow no longer applies to Alice
  - The same request is implicitly denied because no matching Allow remains
- Alice's identity remains but the permission relationship has changed

IAM Policies

- An IAM policy is a document that expresses permissions
  - JSON, a structured text format, represents the policy elements

- A statement describes an effect, an action, and a resource
  - Effect is Allow or Deny
  - Action names the operation
  - Resource identifies its target

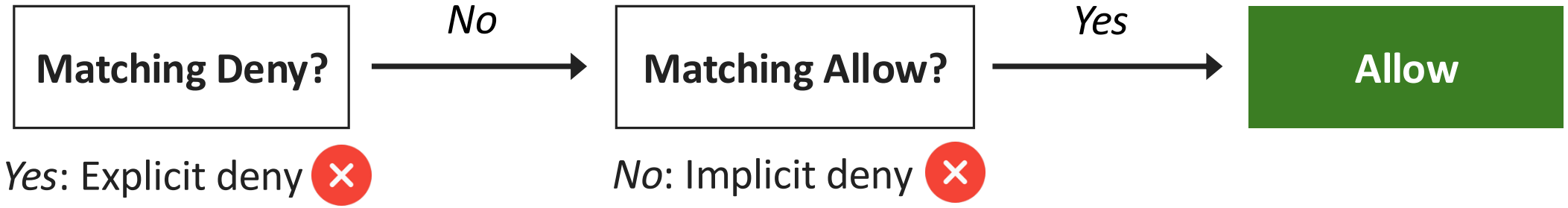
- *Attachment* determines which identity receives an identity-based policy
  - A policy contains permission rules – not passwords or access keys

Reading a Policy

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "iam:ListUsers",
    "Resource": "*"
  }]
}
```

- This policy permits *iam:ListUsers* (listing IAM user)
  - Effect is “Allow”
  - It does not permit creating, changing, or deleting users
- Resource is “*”
  - Required for this action; does not permit all actions
- Attach this policy to the Readers group to make the permission available to its members

Policy Evaluation



- Example scope: identity-based policies, with no additional restrictions
- A matching explicit Deny overrides an Allow
- Without an explicit Deny, a matching Allow permits the request
  - If no Allow matches, the result is an implicit deny
- Other policy types can further restrict real requests

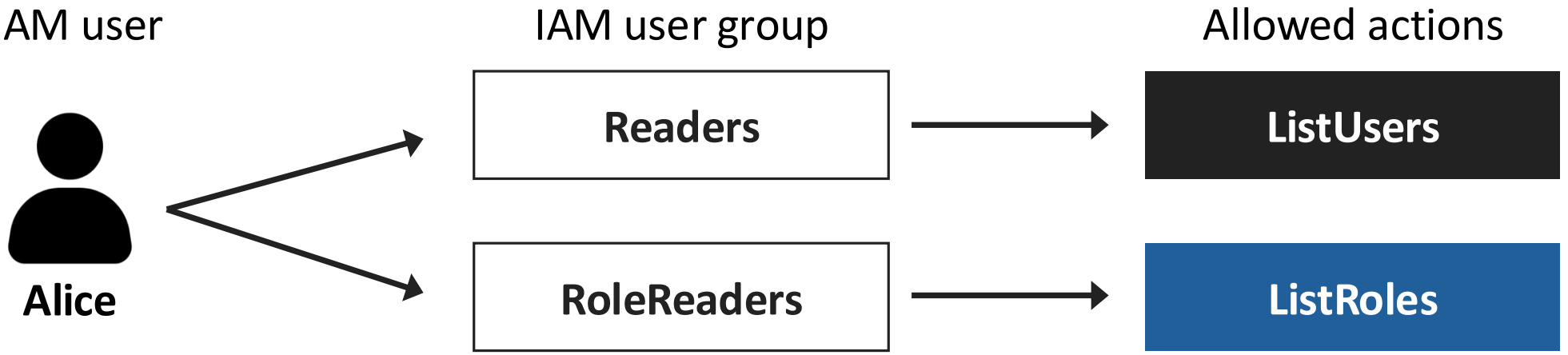
Permission Outcomes

Assume Alice has the policy statements shown below.
Each row represents a separate scenario.

Case	Policy statements applied to Alice	Alice's request	Result
1	Allow ListUsers	ListUsers	Allowed
2	Allow ListUsers	CreateUser	Implicit deny
3	Allow ListUsers + Deny ListUsers	ListUsers	Explicit deny

- Case 1: A matching Allow permits the request
- Case 2: No matching Allow → Implicit deny
- Case 3: An explicit Deny overrides an Allow

Combining Permissions



- Alice belongs to Readers and RoleReaders
  - Readers grants ListUsers
  - RoleReaders grants List Roles
- Permissions from both groups are combined
  - Alice can perform both ListUsers and ListRoles
- If Alice leaves RoleReaders, she loses that source of ListRoles permission

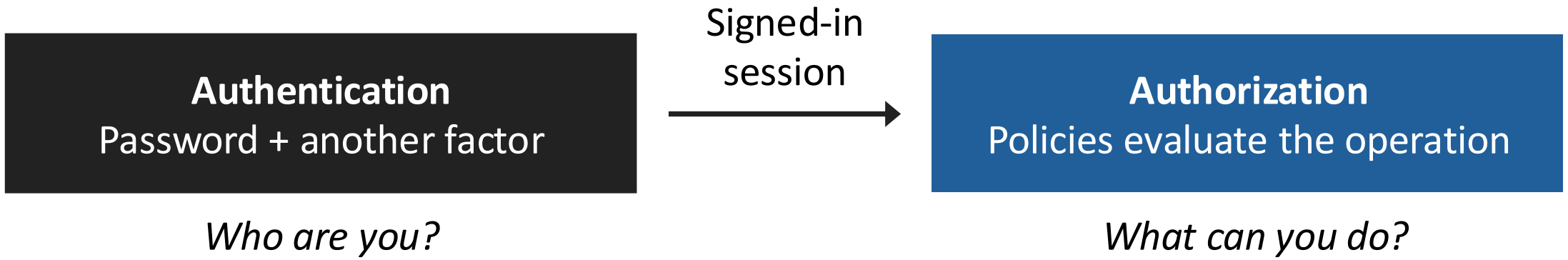
Least Privilege

- Identify the tasks that an identity needs to perform

- Grant only the permissions required for the task
  - ex) If Alice only needs to list IAM users, grant iam:ListUsers, not broad IAM read-only permissions such as iam:ListRoles
  - Even read-only access can expose unnecessary information

- Check effective permissions across all applicable policies
  - Permissions may come from multiple policies or group memberships
  - Remove unnecessary permissions and verify that required tasks still work

MFA and Permissions



- Multi-factor authentication (MFA) requires more than one type of authentication evidence
  - It reduces the risk from a stolen password alone
- Policies still determine what the authenticated session may do
  - MFA does not remove excessive permissions or prevent all session theft
- Use stronger authentication together with least privilege

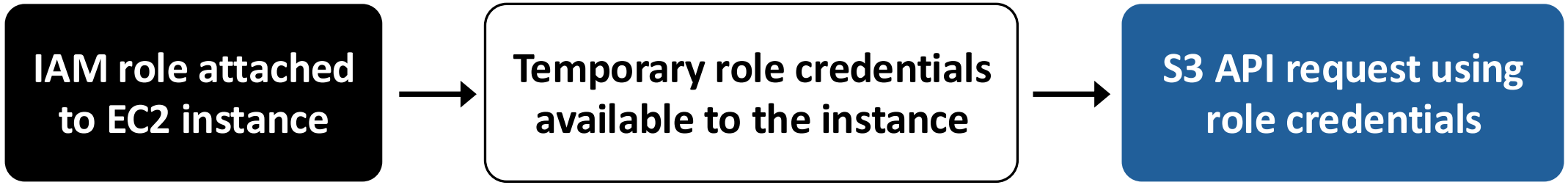
Workload Credentials

- An application needs credentials when it makes authenticated AWS requests
- Hardcoded long-term keys can leak through code, configuration, or compromised hosts
  - A usable copy can authenticate requests outside the original application



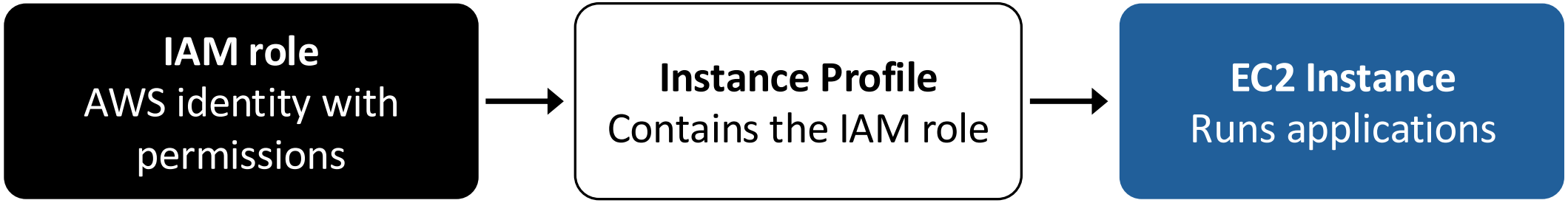
- IAM roles provide temporary credentials for workloads
  - Limited lifetime reduces exposure
  - Appropriate permissions are still required

IAM Roles



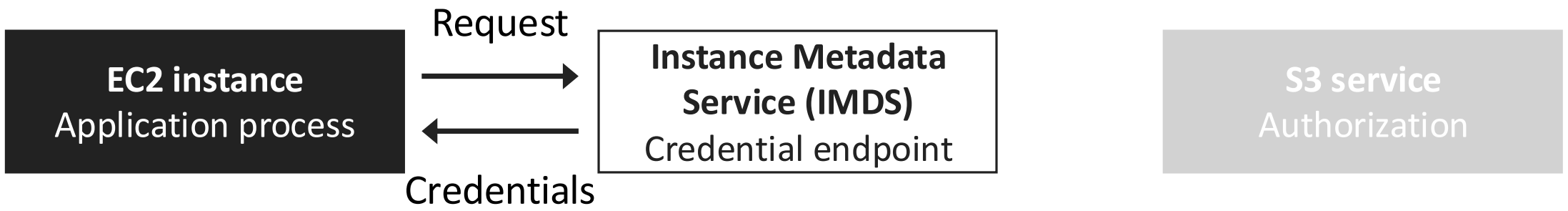
- An IAM role is an identity with permissions that trusted principals can assume
  - Assuming a role creates a session with temporary credentials
- For EC2, a role is associated with the instance
  - Processes able to access its credentials can use the role's permissions
- The role is not a password or a policy document

Associating an EC2 Role



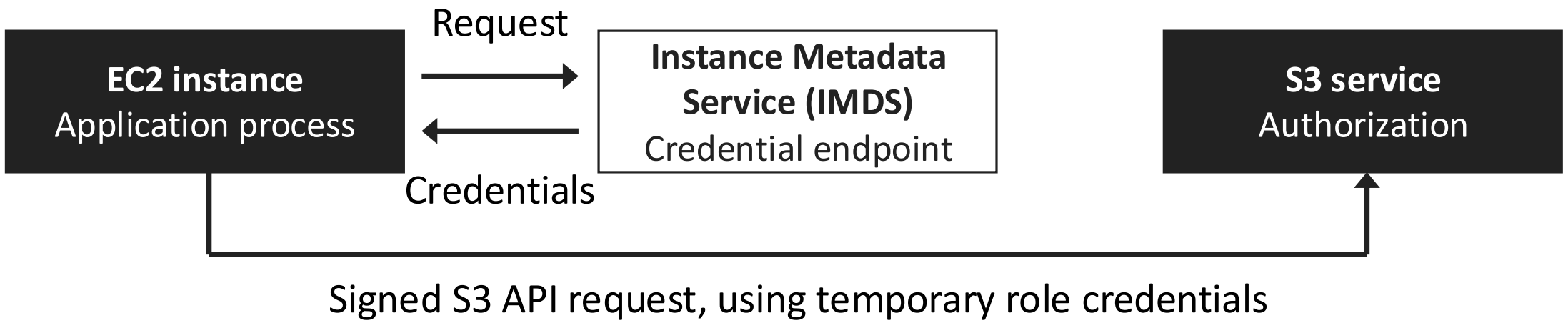
- An instance profile is a container used to associate an IAM role with an EC2 instance
  - AWS Console automatically creates an instance profile when creating an EC2 role
- An authorized administrator associates the instance profile with the EC2 instance
- Once associated, temporary role credentials can be made available to applications running on the instance

Obtaining Temporary Credentials



- The Instance Metadata Service (IMDS) provides instance information to software running on EC2
  - Its credential endpoint provides temporary credentials associated with the instance's IAM role
- An AWS SDK (Software Development Kit) can automatically retrieve these credentials
  - Temporary credentials are issued as a set of three values: an access key ID, a secret access key, and a session token
  - The application does not store a permanent access key

Making an AWS Request



- The SDK signs the API request using the temporary credentials
- S3 authenticates the request and evaluates the applicable permissions
  - A valid credential set does not permit every S3 operation
  - The requested action and object must satisfy the authorization rules

Trust and Permissions

Role Trust Policy
Which principal may assume the role?

ex) EC2 service (*ec2.amazonaws.com*)

Role Permissions Policy
Which operations may the session perform?

ex) Read S3 objects (*s3:GetObject*)

- The trust policy controls which principals may assume the role
  - In our example, the EC2 service is trusted to assume the role
- Permissions policies specify which AWS operations the role can perform
  - ex) the role may be allowed to read specific S3 objects
- Trusting EC2 does not automatically grant access to S3 resources

Role Trust Policy

Example: ReportReaderRole for an application running on EC2

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Service": "ec2.amazonaws.com"
    },
    "Action": "sts:AssumeRole"
  }]
}
```

- An IAM role can be assumed by trusted principals, such as IAM users, other roles, or AWS services
- A trust policy specifies which principals are allowed to assume the role
- In this example, ReportReaderRole trusts the EC2 service (ec2.amazonaws.com)
  - AWS can provide temporary role credentials to EC2 instances associated with this role
- The trust policy defines who can assume the role, not what the role can do

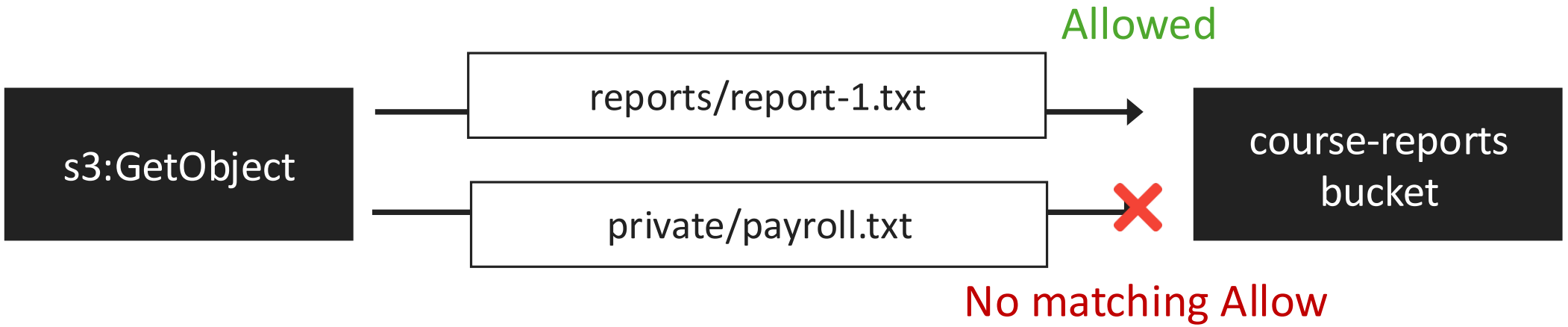
Role Permissions Policy

Same IAM Role: ReportReaderRole

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource":
     "arn:aws:s3:::course-reports/reports/*"
  }]
}
```

- A permissions policy defines which AWS operations the role is allowed to perform
- Action: s3:GetObject allows reading objects from Amazon S3
- Resource specifies which objects can be accessed
  - arn:aws:s3:::course-reports/reports/* selects objects under the reports/ prefix
- This policy does not allow deleting objects or reading objects outside the specified prefix

Role Permissions Policy – Resource Scope



- The Resource field limits which S3 objects the role can read
- reports/* matches object keys beginning with reports/
  - An object key is the name of an object within an S3 bucket
- Reading reports/report-1.txt is allowed, while reading private/payroll.txt is not allowed by this policy
- Other applicable policies may affect the final authorization decision

Policy Attachment

Identity-based policy

- Attached to an IAM identity (e.g., ReportReaderRole)
- Defines what actions the identity can perform on which resources
- ex) the role permissions policy from slide 22

Resource-based policy

- Attached to an AWS resource (e.g., an S3 bucket)
- Defines which principals can access the resource and what actions they can perform
- ex) An S3 bucket policy allowing ReportReaderRole to read objects

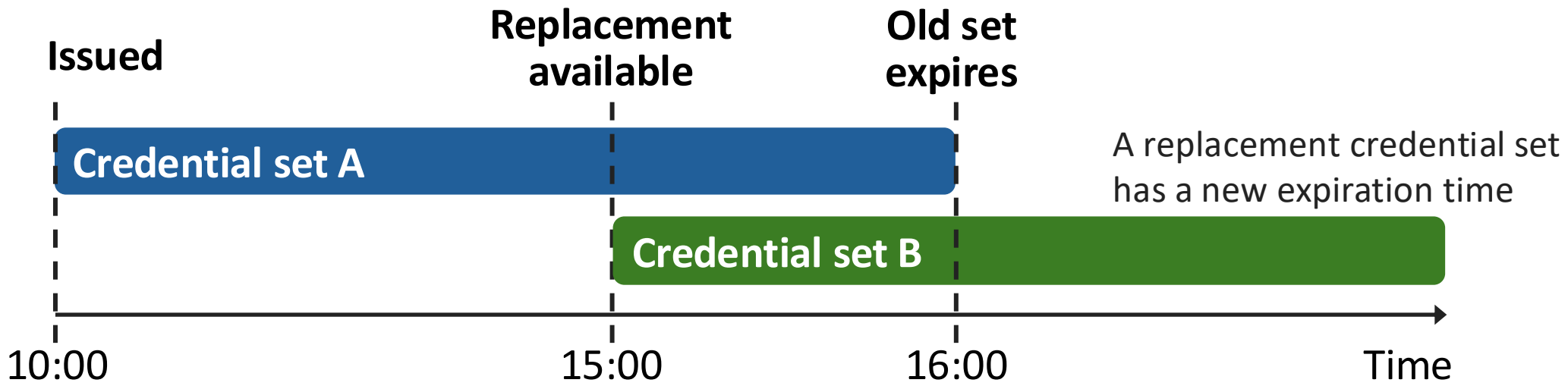
- Access to S3 can be granted through identity-based or resource-based policies
  - For same-account access, either policy type may grant access

Testing Report Access

- Testing the permissions of ReportReaderRole from the previous slides
- Assume the role policy is the only applicable grant, with no additional restrictions
- The role policy allows s3:GetObject only for objects under reports/*
  - Test the action boundary and the resource boundary separately

Action	Object key	Expected result
GetObject	reports/report-1.txt	Allowed
GetObject	private/payroll.txt	Implicit Deny
DeleteObject	reports/report-1.txt	Implicit Deny

Credential Lifetime



- Each temporary credential set has an expiration time
  - Requests using that set fail after it expires
- AWS makes new role credentials available through IMDS before the old credentials expire
  - The AWS SDK can automatically retrieve and use the new credentials

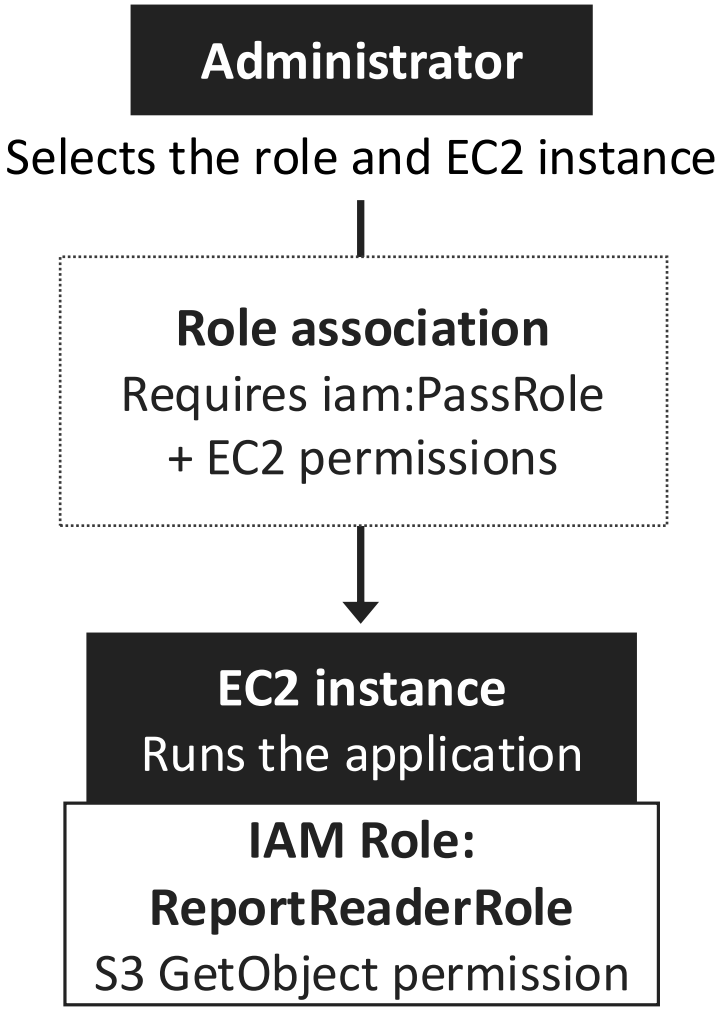
Credential Exposure

- Stolen temporary credentials can be used until they expire
- The role's permissions limit what an attacker can do

- One-time credential theft
  - Expiration limits the lifetime of the stolen credential set
- Persistent instance compromise
  - An attacker with continued access to IMDS may obtain new credentials

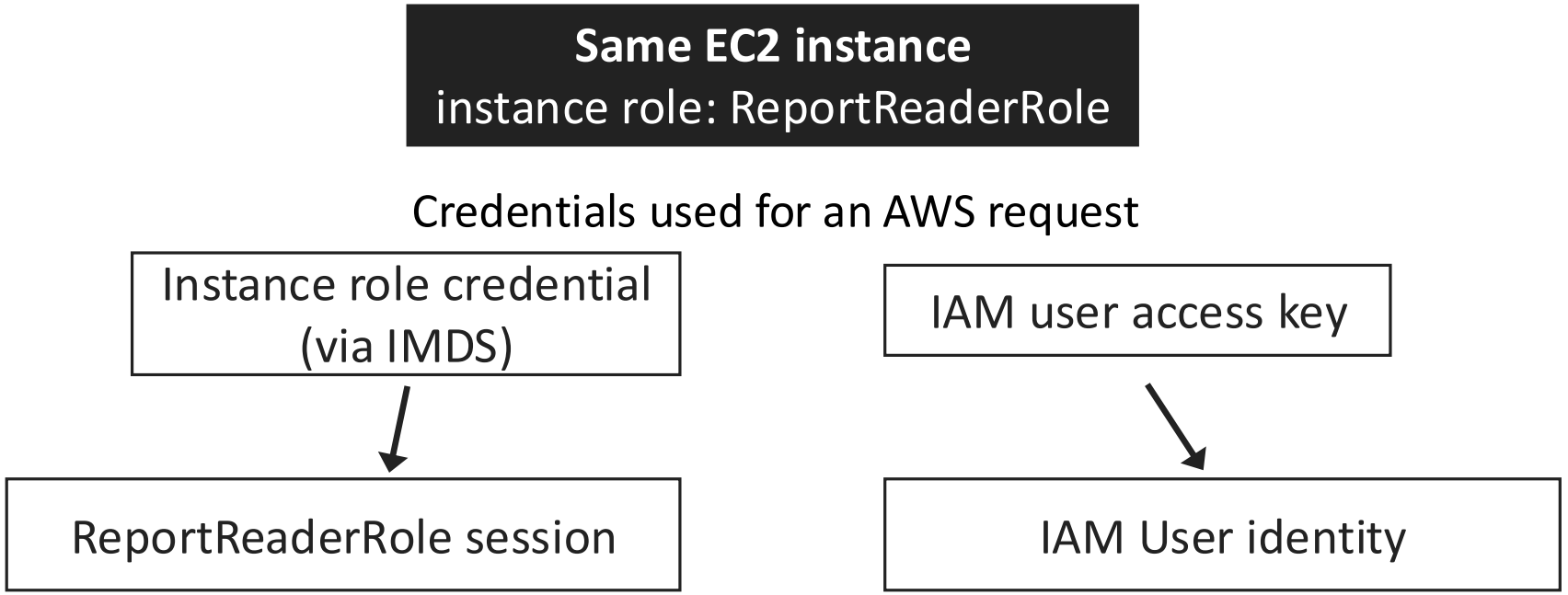
- Temporary credentials reduce exposure, but do not eliminate the risk of credential theft

Role Administration



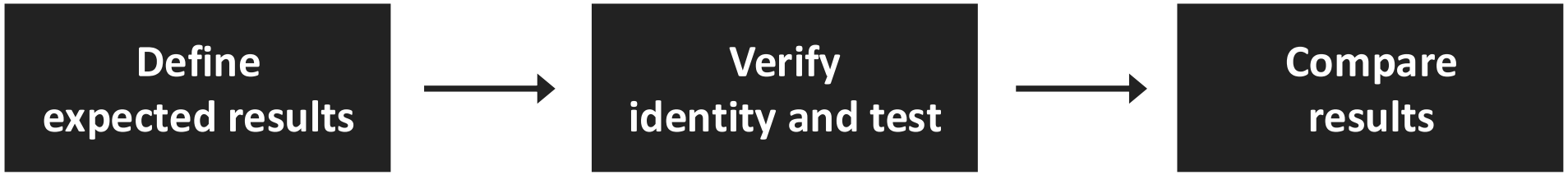
- An administrator can associate an IAM role with an EC2 instance
- This operation requires iam:PassRole and the necessary EC2 permissions
- iam:PassRole controls which roles the administrator is allowed to assign to AWS services

Request Identity



- AWS identifies the caller based on the credentials used for an API request
- An application running on EC2 may use instance role credentials or explicitly configured IAM user credentials

Verifying Permissions



- Define which requests should be allowed or denied
  - Ex) Verify that ReportReaderRole can read reports/* but not private/*
- Verify the caller's identity and test both allowed and denied requests
- Compare the actual results with the expected outcomes
- After changing a policy, repeat the same tests to verify the effect

Summary

- AWS IAM controls access to AWS resources through identities and policies
- IAM users and groups manage human permissions, while roles provide temporary credentials for trusted principals
- Trust policies define who can assume a role; permissions policies define what the role can do
- Apply least privilege and verify permissions by testing allowed and denied requests
- **Next session: AWS lab**
  - **Please bring your laptop**